

Growth Engine

Desired Reasoning Layer, Learning Database and Visual Graph

Internal product note | May 2026

Core point

As it appears to be structured now, the visual graph is, in my view, not yet how the product should ultimately work. If the graph mainly shows that a new ad is based on an old winning ad or on loose metrics such as ROAS, CPC or CTR, then that is too flat. Growth Engine should show that the system learns underlying relationships from historical data, contextual data and performance data. A new ad should therefore not be created directly from an old ad, but from a living relationship database that continuously evolves.

1. Why the current interpretation is too limited

A simple visual graph that says new ad A is based on old ad 7 is useful for explanation, but it does not show the real value of Growth Engine. In that version, the system mainly looks like a variation generator: it finds a winner and creates a new variant of it.

That is not wrong as a feature, but it is not the core of the product. The core should be that Growth Engine learns which underlying factors are connected to better results. The old ads are not the direct template; they are the evidence behind a learned relationship.

Important distinction

Old ads are evidence. The learned patterns are the strategy. The new ad is generated from that strategy, not simply from a copied winner.

2. How Growth Engine should work in my view

The system should consist of multiple layers. Each layer has a different role. The LLM or image generator should only come at the end, after the reasoning layer has determined which direction is logical based on data.

Layer	What is inside?	Role in the system
1. Historical data layer	Raw ads, Meta results, Shopify results, product data, audience, date, season, possible weather data and other context.	This is the evidence. No creative conclusion should be drawn from this without interpretation.
2. Feature layer	Attributes such as hook type, visual style, product category, audience, offer, price level, season, colour use, UGC/lifestyle/product photo.	The system translates raw ads into comparable building blocks that can be reasoned over.
3. Relationship layer	Learned patterns such as: summer visuals work better above 20 degrees for accessories; urgency drives high CTR but lower Shopify conversion.	This is the real learning database. PLN/MeTTa/OmegaClaw should form relationships here, maintain confidence and store evidence/counter-evidence.
4. Decision layer	The user request, product, audience, timing, objective and relevant learned patterns.	The system searches which learned relationships are relevant for this new task.

5. Generation layer	Ad copy, visual prompt, image edit, layout and export.	The LLM/image generator executes the creative output, but should not invent the strategic reason itself.
---------------------	--	--

3. Example: summer context, beach loungers and sunglasses

A good example is not: “ad 19 performed well, so make something that looks like ad 19.” A good example is that the system learns a relationship across multiple campaigns:

Example of a learned pattern	When the temperature is above 20 degrees and the product is a lifestyle/accessory product, summer visuals with beach context, sunglasses, light colours and relaxed social-proof hooks have historically performed better. This relationship is supported by ads 2, 6, 7 and 19, including ROAS, CPC, CTR, spend and Shopify conversion. The system also stores exceptions and counter-evidence.
------------------------------	--

After that, when a user asks for a new ad for a comparable product in June or July, the system should not randomly copy an old winner. It should first retrieve relevant learned patterns and derive a creative strategy from them.

4. Desired flow when generating a new ad

Step 1: Understand the user request — For example: “Create a new ad for this bracelet for June, targeting a cold Meta audience.”

Step 2: Determine context — Product category, price level, season, audience, historical performance and possible weather/context data are retrieved.

Step 3: Search relevant learned patterns — The system searches the relationship database for patterns that match the product, timing, audience and objective.

Step 4: Formulate the strategy — For example: use a summer/lifestyle visual, social-proof hook, light colour palette and no aggressive discount.

Step 5: Generate the creative execution — Only now do the LLM and image generator create the concrete copy and visual, within the boundaries of the selected strategy.

Step 6: Show the trace — The user sees which patterns were used and which historical ads/data form the evidence.

Step 7: Feed results back into the system — New performance data is added to the Atomspace/learning database. Confidence rises, falls or the pattern is adjusted.

5. How the visual graph should look

The graph should not only show “new ad -> old ad”. The graph should show the reasoning in layers: from evidence to feature, from feature to pattern, and from pattern to new ad.

Graph level	What the user sees	Why this matters
Raw evidence	Old ads, metrics, date, audience, product and context.	Shows that the pattern is not invented out of thin air.
Extracted features	Hook type, visual style, format, offer, season, product category.	Makes clear which attributes the system extracted from the ads.

Learned pattern	For example: “summer lifestyle visual + warm weather + accessory product -> higher ROAS probability.”	This is the real reasoning layer and the difference from ordinary AI ad generation.
Confidence and evidence	Confidence score, number of ads, spend, average metrics, counter-evidence.	Prevents weak patterns from being presented as hard truth.
New ad decision	Why this new ad uses this hook, visual and CTA.	Connects the data reasoning to the creative output.

Ideal graph structure	Historical ads -> extracted features -> learned patterns -> confidence/evidence -> new ad strategy -> generated ad. The old ads remain visible, but as evidence underneath the learned pattern. They are not the only source of the new ad.
-----------------------	---

6. What this means technically for the developer

The current pipeline may have a winner-to-variation loop, but that should not be the only or primary intelligence. There needs to be a separate layer in which relationships are stored and revised as first-class objects.

LearnedPattern objects: Make patterns explicit objects with a name, conditions, predicted outcome, confidence, evidence and counter-evidence.

Evidence links: Every pattern must link back to the old ads, performance rows and contextual data from which it was created.

Confidence with substantiation: Confidence should not just be a nice-looking number. It should depend on sample size, spend, consistency, variance and counter-evidence.

Not only ROAS/CPC: Use metrics as evidence, but combine them with context: season, product, audience, offer, visual style and Shopify conversion.

Pattern retrieval before generation: When a new ad is requested, the system must first retrieve relevant patterns before an LLM or image model is called.

The prompt should not invent the strategy: The prompt to the LLM should already contain the strategy. The LLM may only execute copy/visuals within that strategy.

Update after new results: New results must strengthen, weaken, split or replace existing patterns. The learning database must therefore evolve.

8. How we should explain this as product vision

Product explanation	Growth Engine does not generate new ads by simply copying old winners. The system first builds a living relationship database from historical ads, performance data, Shopify results and context. PLN/MeTTa/OmegaClaw uses that database to learn patterns with evidence and confidence. When a user requests a new ad, the system searches for the most relevant learned relationships for that product, season, audience and objective. The generated ad is therefore based on learned patterns, not loose examples. Every pattern can be traced back to the old ads and data from which it emerged.
---------------------	--

9. Checklist: when does it work as intended?

- ☐ Can we show which learned patterns were used for every new ad?
- ☐ Can every pattern be traced back to concrete historical ads and raw performance data?
- ☐ Are contextual factors included, such as season, product type, audience and possibly weather?
- ☐ Can the system mark weak patterns as uncertain instead of presenting them as truth?
- ☐ Is the LLM only called after the strategy comes from the reasoning layer?
- ☐ Are new results written back to the relationship database?
- ☐ Can a pattern become stronger, weaker or adjusted after new data?
- ☐ Can we explain why a new ad was created without the LLM having to invent the answer itself?

10. Conclusion

The visual graph should therefore not only show an inspiration path. It should prove the real product promise: Growth Engine learns relationships from data, uses those relationships for new creative decisions and continues to improve itself after new results.

If we build this well, Growth Engine is not an ordinary AI ad generator and not a simple winner-variation tool. It becomes an account-specific learning engine that increasingly understands which creative choices work in which context.